



# MARCONI: PREFIX CACHING FOR THE ERA OF HYBRID LLMS

**MLSYS'25**

**Rui Pan, Zhuang Wang, ZhenJia, CanKarakus,  
LucaZancato, Tri Dao, Yida Wang, RaviNetravali**

**汇报人：江建谕  
2026.5.25**

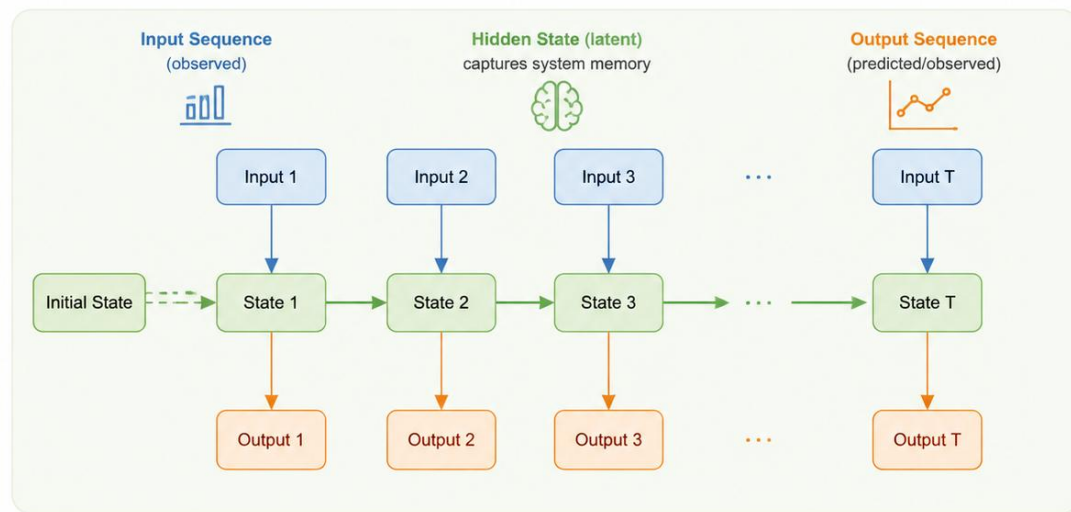
# 目录

- **研究背景与动机**
- **相关工作**
- **设计实现**
- **实验评估**
- **总结与后续思考**

# 研究背景

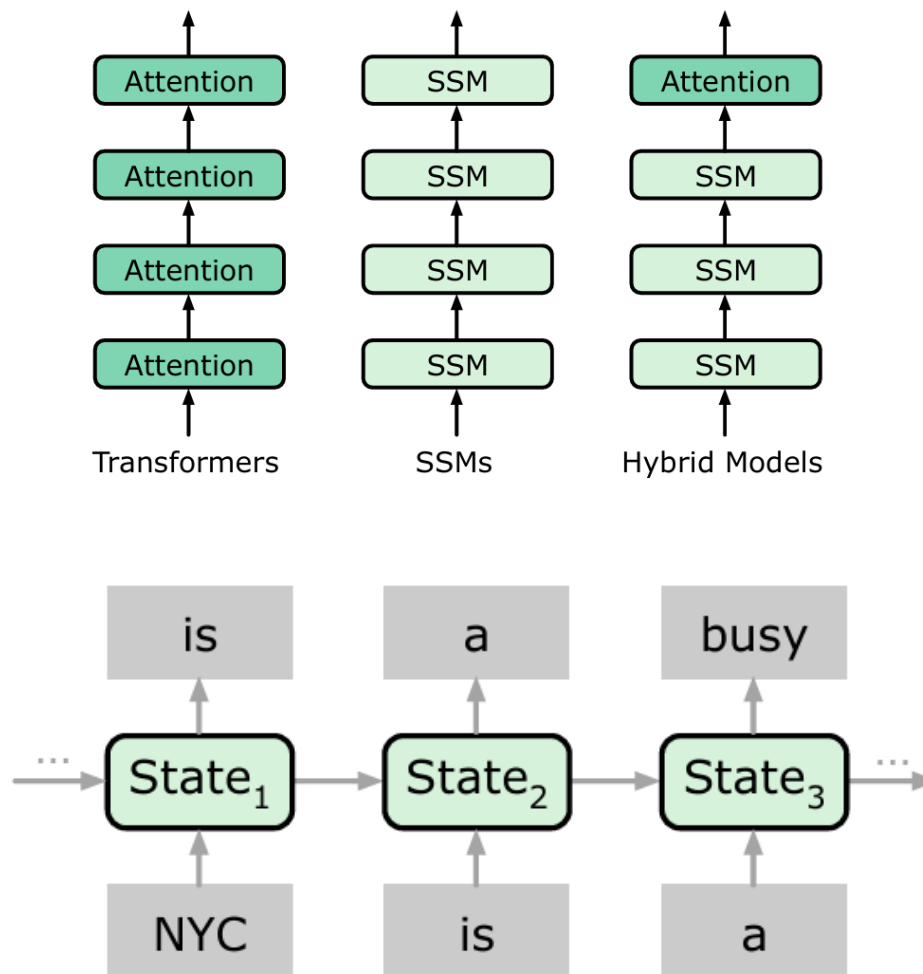
- **Transformer:**
  - 基于自注意力, 每个 token 与所有 token 交互  
→ 计算复杂度  $O(L^2)$
  - 推理时需缓存 KV, 大小; 随序列长度  $L$  线性增长  
→ 长序列显存压力大
- **状态空间模型 (SSM)**
  - 如 Mamba
  - 用固定大小的状态压缩全部历史信息
  - 每来一个新 token, 状态原地更新 (覆盖旧状态)
  - **优势:**
    - 计算复杂度  $O(L)$  (线性)
    - 显存占用 恒定, 不随序列长度增长
- **纯 SSM 的局限:** 在强召回、复杂上下文学习任务上性能不及 Transformer

|       | Attention | SSM    |
|-------|-----------|--------|
| Time  | $O(L^2)$  | $O(L)$ |
| Space | $O(L)$    | $O(1)$ |



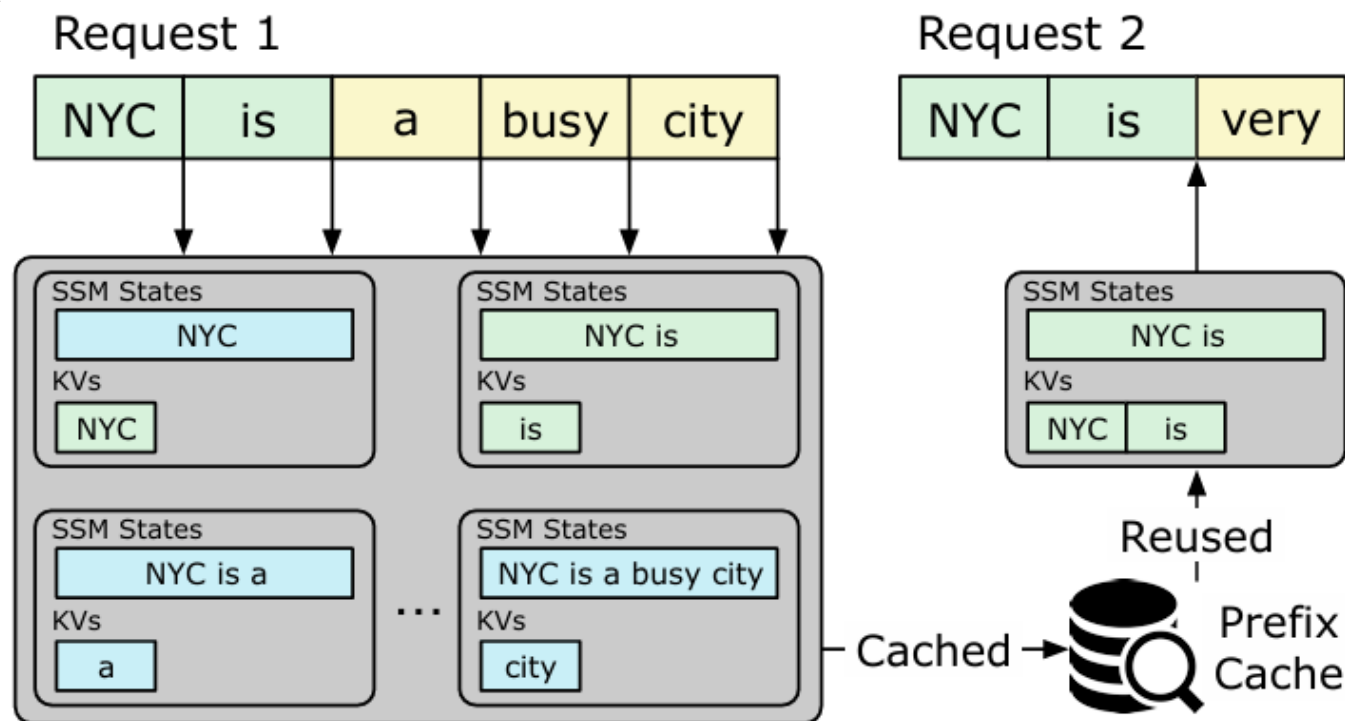
# 研究背景

- **动机：**结合 Attention 的强能力与 SSM 的高效率
- **架构：**Attention 层与 SSM 层按比例交错 (常见 1:6 或 1:8)
- **代表模型：**Jamba, Zamba, Samba
- **性能：**相比同规模 Transformer，训练更快、推理更高效，同时保留较好质量
- **新挑战：**
  - 前缀缓存需要同时管理 KV (来自 Attention) 和 SSM 状态
  - SSM 状态具有“全有或全无”特性 → 传统缓存直接扩展会出现问题



# 研究背景

- **许多请求共享相同前缀**：系统提示词、few-shot 示例、多轮对话历史
- **传统 KV 缓存**：缓存 Attention 层的 K/V 张量，命中后跳过预填充
- **收益**：降低首 token 时延 (TTFT) 提高吞吐
- **现有**：vLLM (PagedAttention) SGLang (Radix Tree)



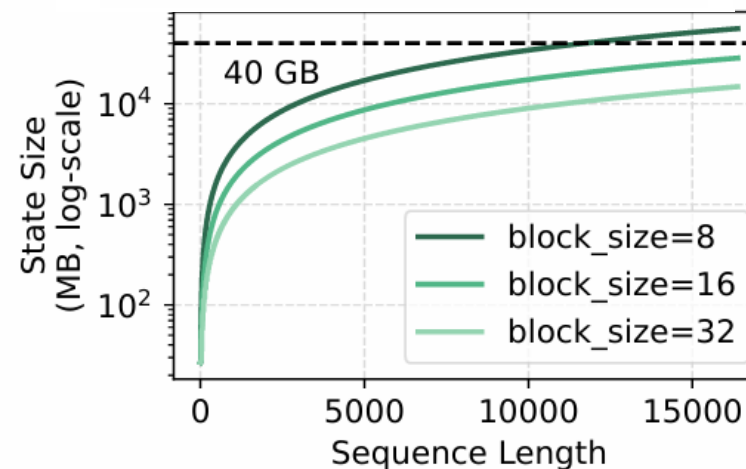
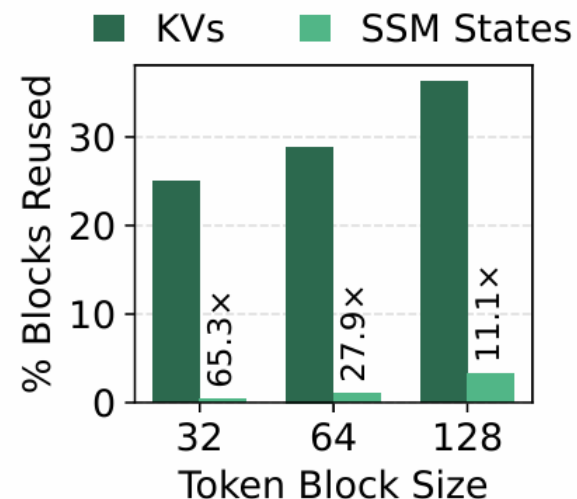
# 研究背景

- **SSM性质**

- 固定大小：与序列长度无关
- 原地更新：无法回滚到前缀状态
- 单状态远大于单 token 的 KV (10~100 倍)

- **后果**

- 要复用前缀，必须保存完全匹配的 SSM 状态
- 细粒度 checkpoint 会产生大量稀疏命中的状态 → 缓存利用率极低
- 显存爆炸



# 相关工作

- **传统前缀缓存 (vLLM, SGLang, Preble, PromptCache) :**
  - 面向纯 Transformer, 所有 token 的 KV 都缓存, 采用 LRU 或 LFU 驱逐
  - 不区分 KV 和 SSM 状态, 直接扩展会导致缓存污染
- **混合模型推理系统:** 现有工作很少, 大多直接复用 Transformer 缓存逻辑
- **Marconi 的定位:** 第一个专为 Hybrid LLM 设计的前缀缓存系统

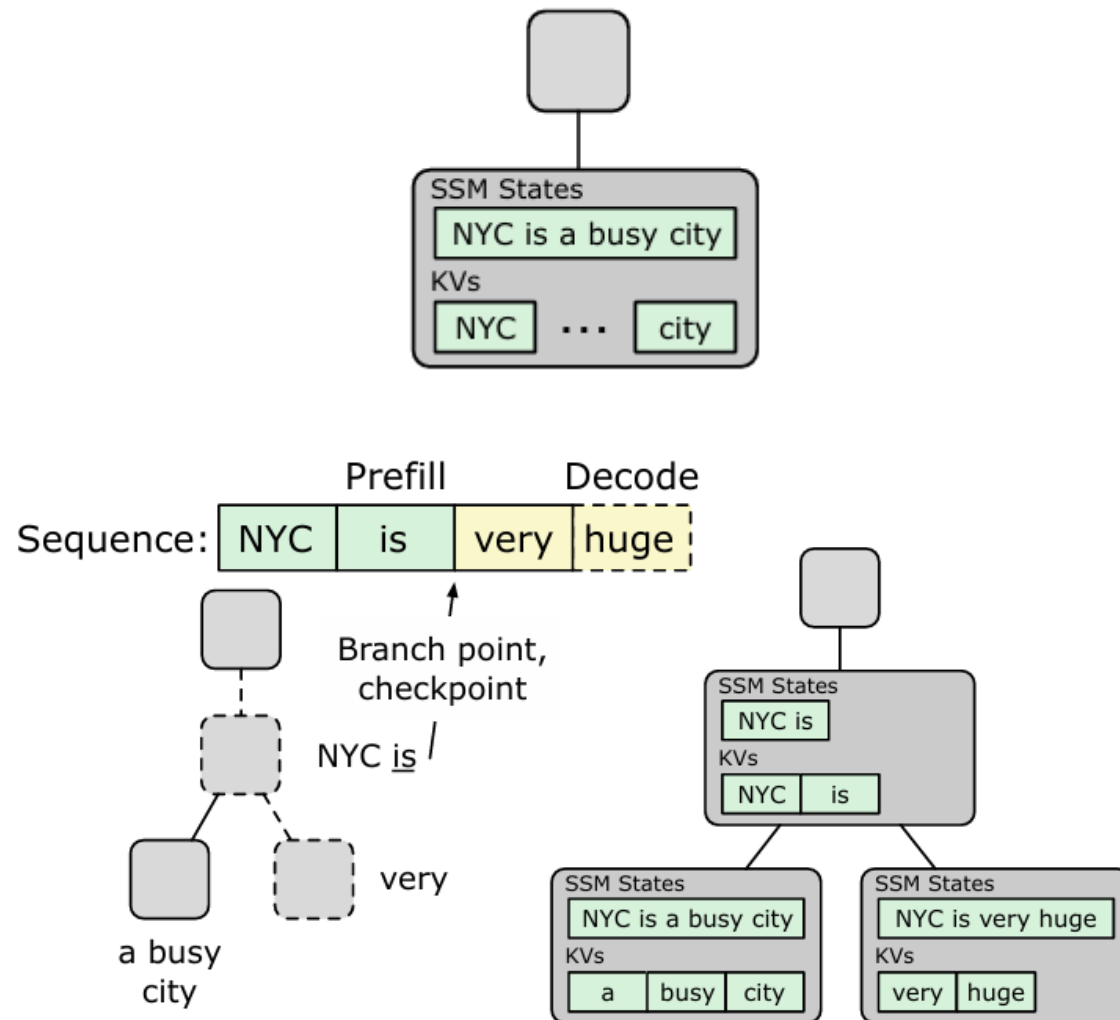
| 系统      | 支持混合模型 | 准入策略 | 驱逐策略       |
|---------|--------|------|------------|
| vLLM    | 拓展支持   | 全量   | LRU        |
| SGLang  | 拓展支持   | 全量   | LRU        |
| Marconi | 原生     | 选择性  | FLOP-aware |





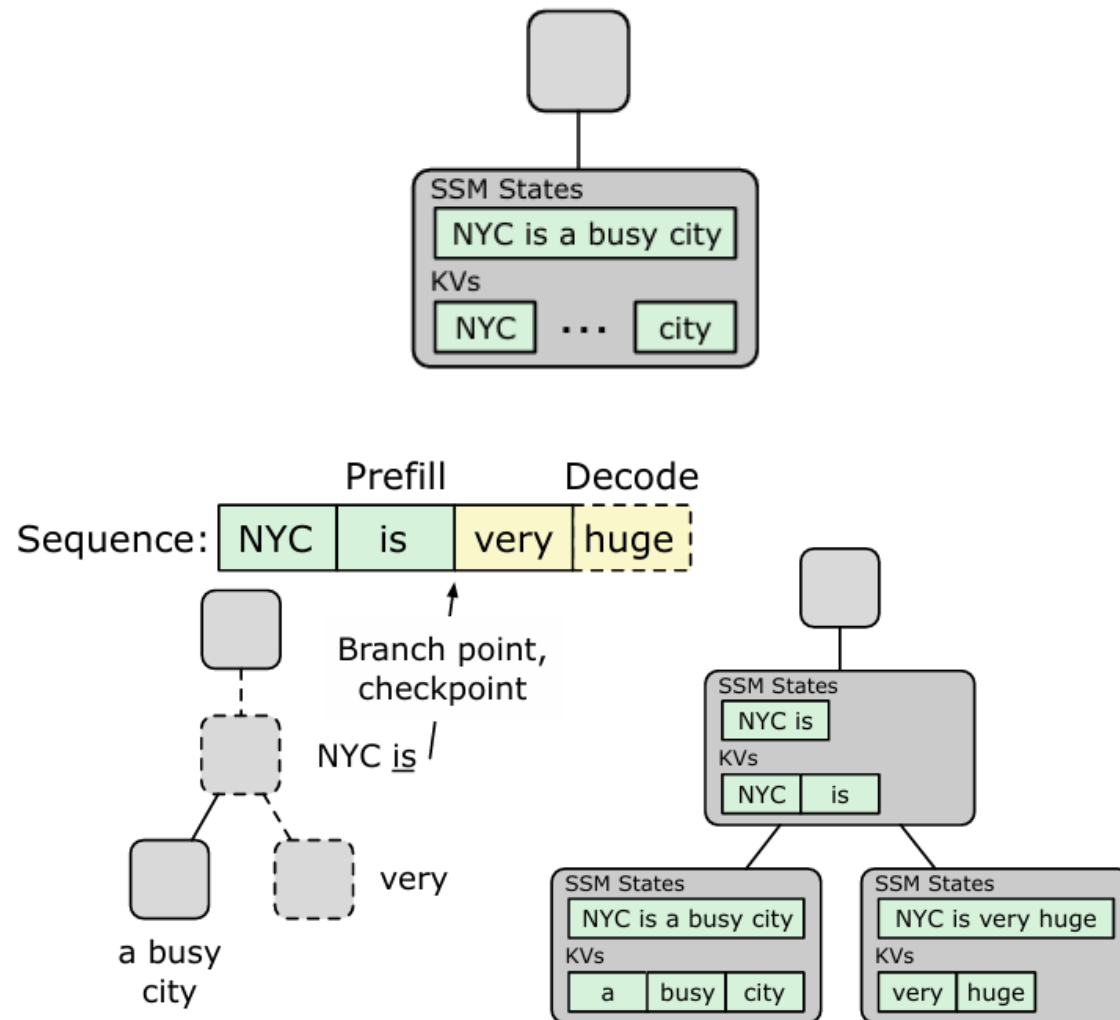
# 设计实现-准入策略

- **纯输入共享 (Purely input shared prefixes) :**
  - 如系统提示词、few-shot 示例
  - 被大量请求复用 → 需要缓存
- **输入+输出共享 (Input+output sharing) :**
  - 如对话历史、agent 轨迹
  - 通常从最后一个解码 token , 而不是从中间分支
- **Marconi 的策略:**
  - 对于纯输入: 在第二次出现时识别并缓存 (投机性插入)
  - 对于输入+输出: 只缓存序列末尾的 SSM 状态



# 设计实现-准入策略

- 使用 Radix Tree 记录历史请求的前缀重叠
- 在预填充前进行 投机性插入 (speculative insertion) :
  - 模拟插入当前序列, 判断是否会产生新的中间节点
  - 如果是, 则将该前缀对应的 SSM 状态 checkpoint
- 获取 SSM 状态的方法:
  - 利用 chunked prefill (如 Mamba2) 缓存 chunk 边界状态
  - 或使用 two-pass prefill (先跑前缀得到状态, 再跑剩余部分)
- 优点: 拒绝大量低效 SSM 状态进入缓存, 避免缓存污染

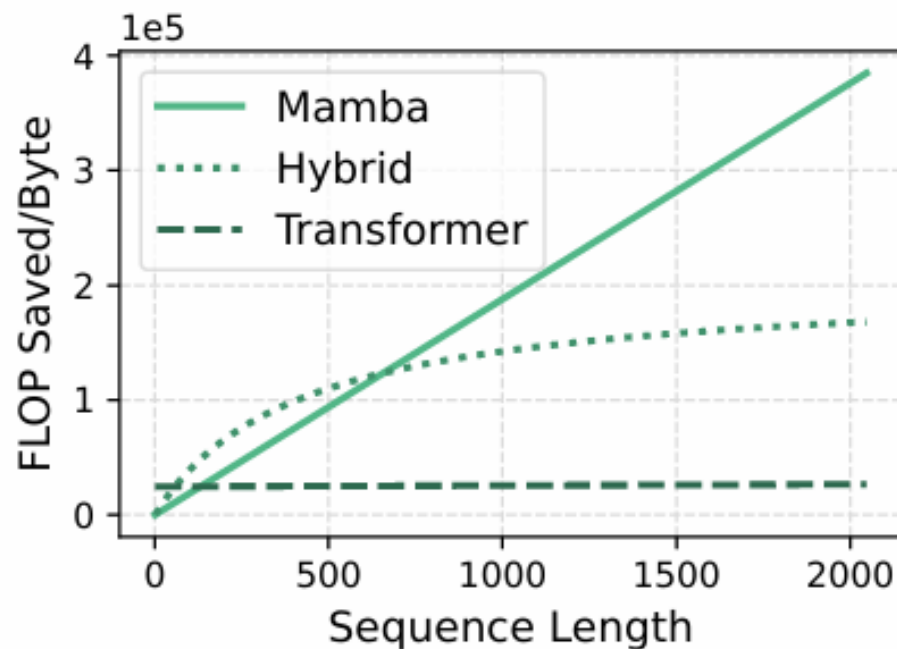


# 设计实现-驱逐策略

- **问题：** KV 大小与序列长度成正比，SSM 状态大小固定，但计算节省都与序列长度相关
- 传统 LRU 只考虑最近使用，无法区分长序列（节省更多 FLOP）和短序列
- **FLOP 效率定义：**

$$\text{FLOP efficiency} = \frac{\text{Total FLOPs saved by reusing this entry}}{\text{Memory consumption of all states}}$$

- SSM 层的 FLOP 效率随序列长度增长极快



# 设计实现-驱逐策略

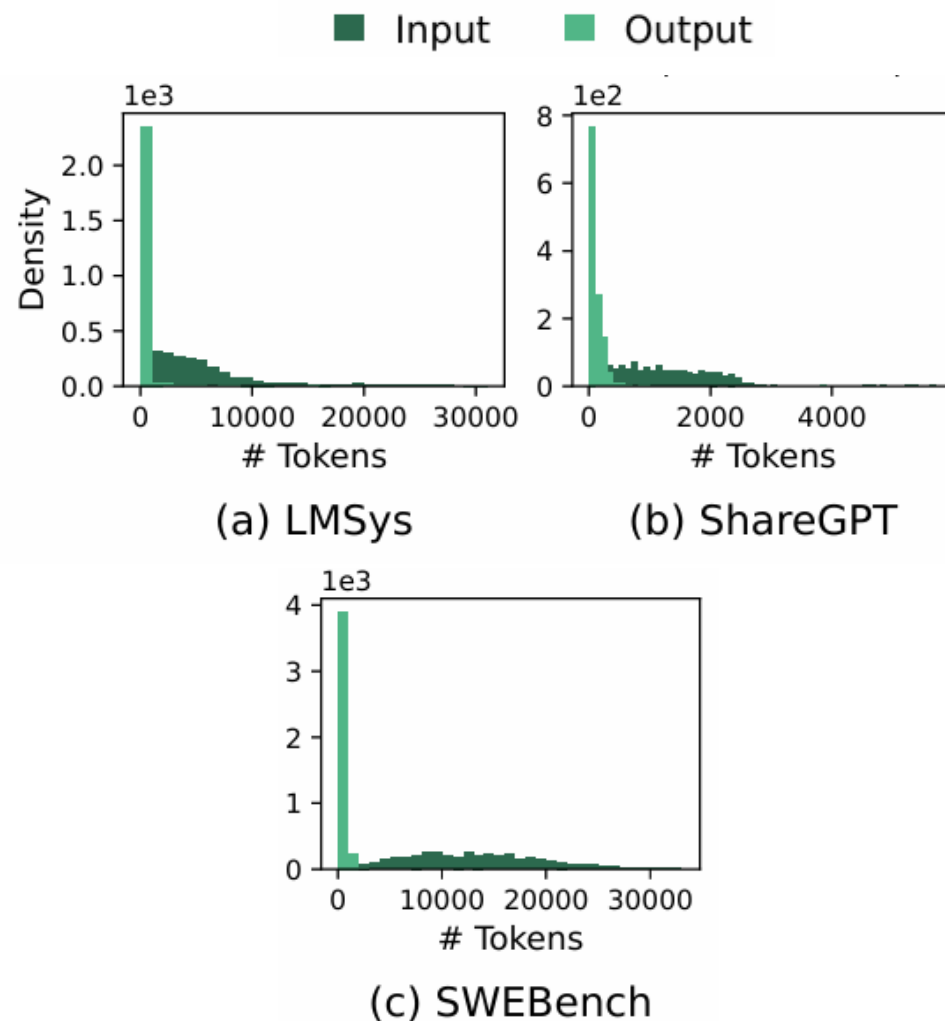
- 每个缓存节点  $n$  的效用分数:

$$S(n) = recency(n) + \alpha \cdot flop\_efficiency(n)$$

- 两者归一化到 (0,1)
- $\alpha$  控制对 FLOP 效率的重视程度
- 自适应  $\alpha$  调优:
  - 启动时用 LRU 收集请求样本
  - 离线网格搜索最优  $\alpha$ , 最大化命中率

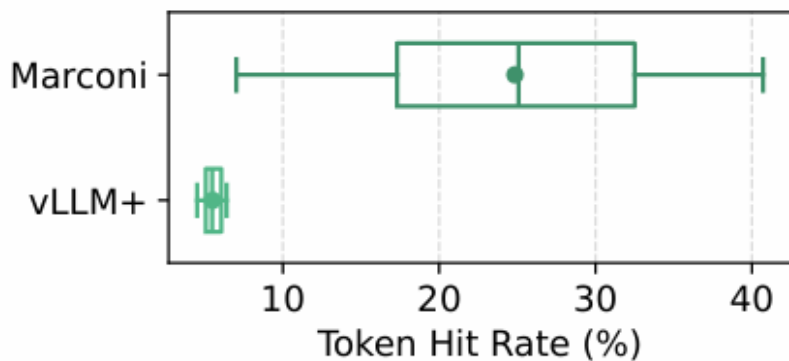
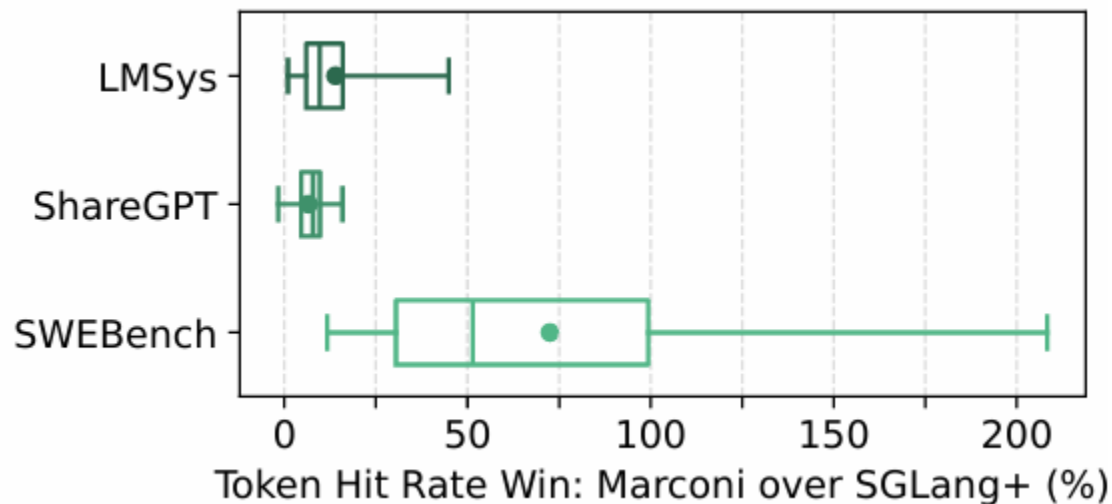
# 实验测试

- **基线:**
  - Vanilla (无缓存)
  - vLLM+ (细粒度 checkpoint, 块大小 32)
  - SGLang+ (Radix Tree + LRU, 同 Marconi 准入策略)
- **模型:** 7B Hybrid (4 Attn, 24 SSM, 28 MLP) , Jamba-1.5-Mini
- **数据集:**
  - LMSys (多轮对话, 长输出)
  - ShareGPT (多轮对话, 短输出)
  - SWE-Bench (agent 轨迹, 极长输入)
- **指标:** Token hit rate, TTFT (P5/P50/P95)

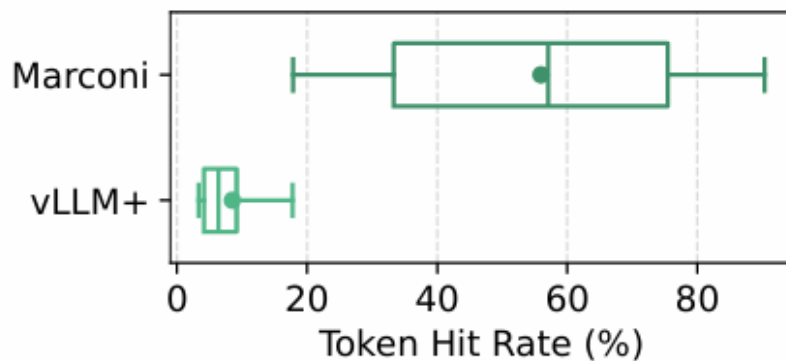


# 实验测试

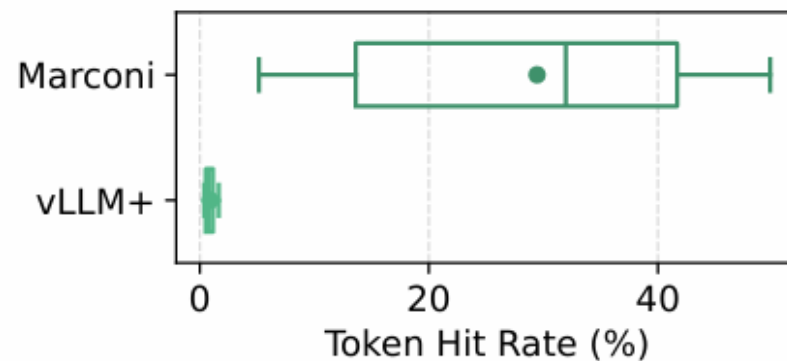
- 与 vLLM+ 对比:
  - Marconi 命中率提升  $4.5\times \sim 34.4\times$
  - 在 SWE-Bench 上提升最大 (长序列多)
- 与 SGLang+ 对比:
  - FLOP 感知驱逐额外提升 19% ~ 220%
  - 长序列场景收益更明显



(a) LMSys



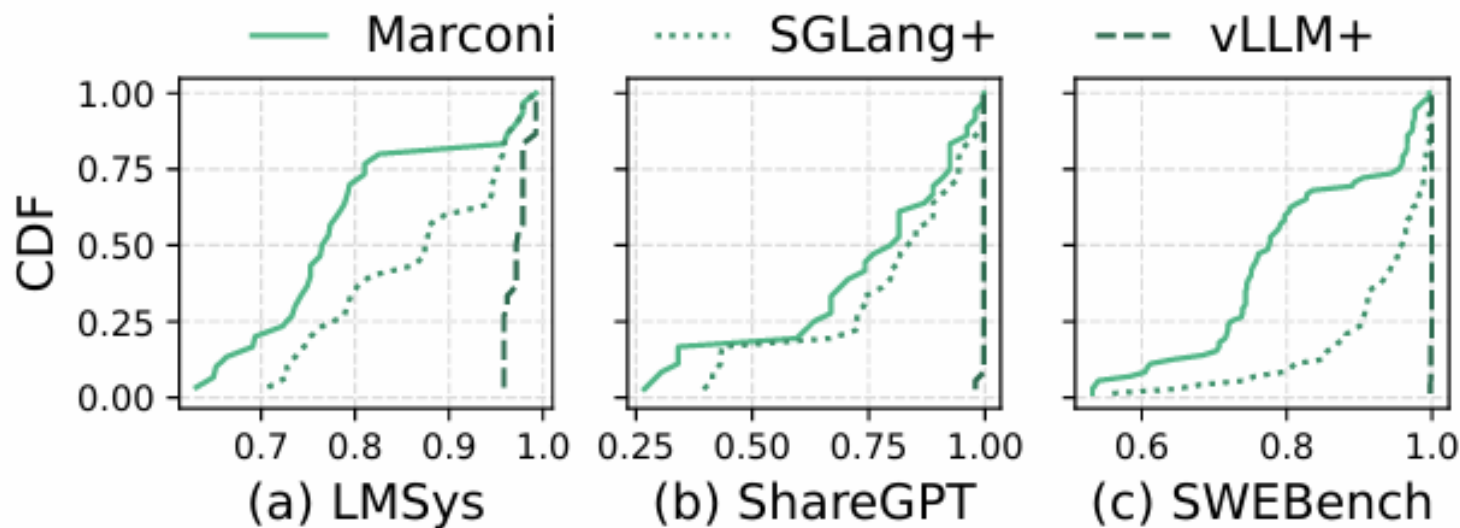
(b) ShareGPT



(c) SWEBench

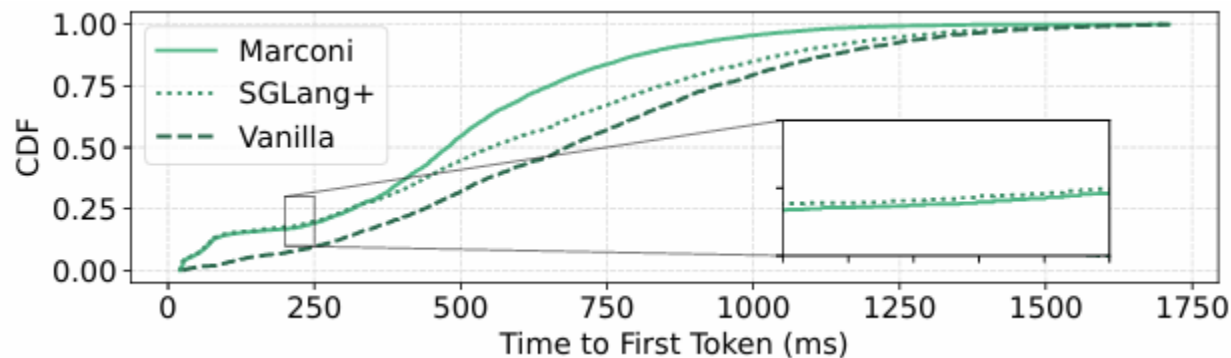
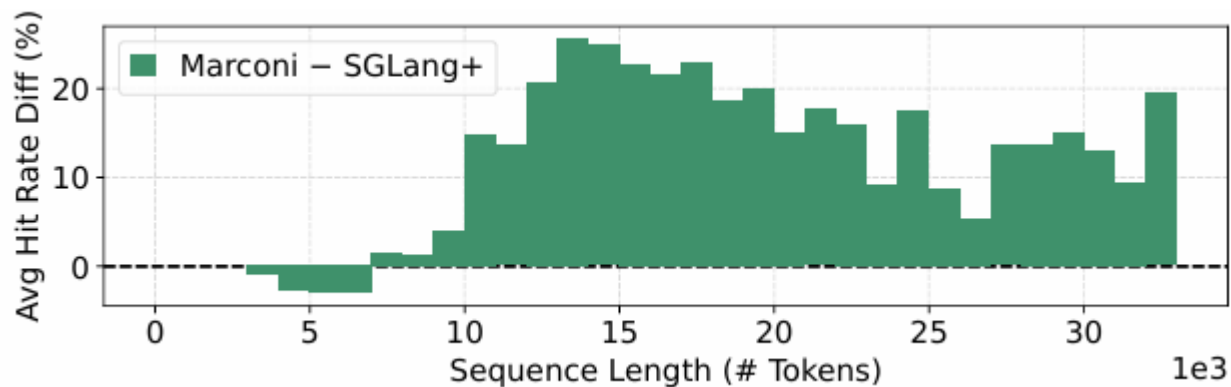
# 实验测试

- P95 TTFT 相比无缓存降低 36~71% (绝对降低 103~617 ms)
- 相比 vLLM+ 额外降低 36~71%
- 相比 SGLang+ 额外降低 13~25%
- 示例: SWE-Bench 上 P95 TTFT 从 1.3s 降到 0.7s 左右



# 实验测试

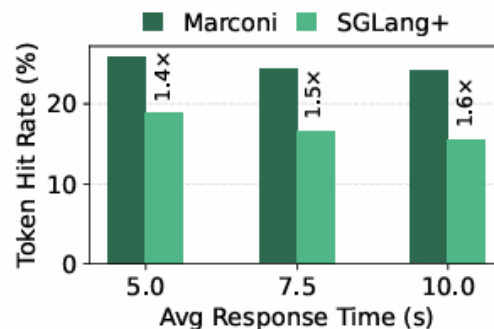
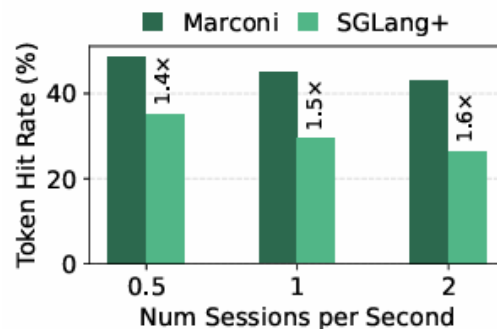
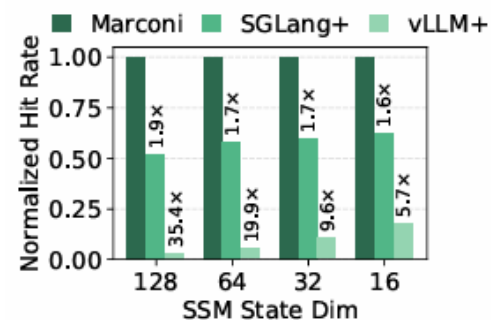
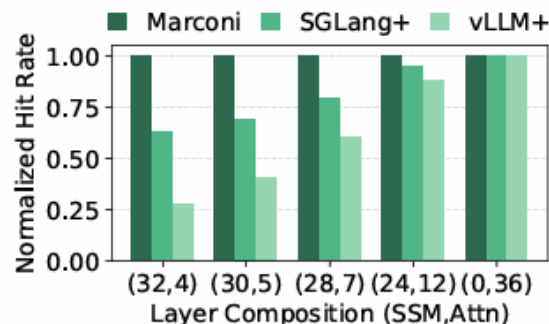
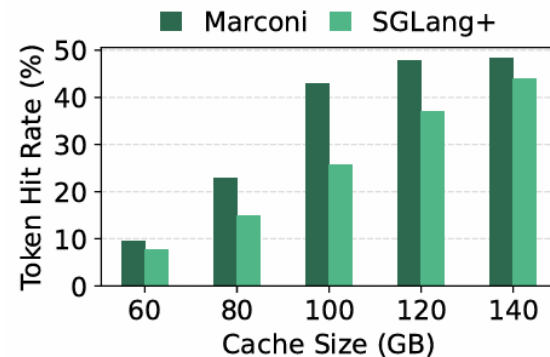
- Marconi 牺牲部分短序列命中率，换取长序列更高命中
  - <7K tokens: 命中率略降 (最多 -3%)
  - 7K tokens: 命中率提升高达 +25.5%
- 总体 FLOP 节省提升 90.3%
- TTFT 影响:
  - P5 略微变差 (+2.1ms, 可忽略)
  - P50/P95 分别降低 13.4% / 22.0%





# 实验测试

- **缓存压力:**
  - 中等压力下收益最大 (命中率提升 68%)
  - 极高压或极低压时收益缩小
- **模型组成:** SSM 比例越高, Marconi 收益越大 (1:2  $\rightarrow$  1:8 时提升从 13.5% 到 2.6 $\times$ )
- **SSM 状态维度:** 维度从 16 增至 128, 提升从 5.7 $\times$  到 35.4 $\times$
- **请求到达率:** 并发越高、响应时间越长, 相对收益越明显



# 总结

- Marconi 是首个面向混合大模型 (Hybrid LLM) 的前缀缓存系统
- 提出 选择性准入 (基于前缀复用分类) 和 FLOP 感知驱逐
- **实验结果:**
  - 命中率最高提升 34 倍
  - P95 TTFT 降低 71% (617 ms)
- **开源:** <https://github.com/ruipeterpan/marconi>

# 后续思考

- **不足**

- 强依赖于前缀的精确匹配
- 在 ‘极高分支且短生命周期’ 负载下的失效问题

- **延伸思考**

- 能否实现 ‘近似状态回滚’ 或 ‘有损状态重用’
- 能否实现 ‘Zero-shot (零样本)’ 的热点前缀预测
- 是否需要更 ‘细粒度’ 的算力评估模型



# Thanks

汇报人：江建谕  
2026.5.18